



Fetch Data with AWS Lambda



Natasha Ong

✓ Executing function: succeeded ([logs](#) )

▼ Details

The area below shows the last 4 KB of the execution log.

```
{
  "email": "test@example.com",
  "name": "Test User",
  "userId": "1"
}
```



Introducing Today's Project!

In this project, we will demonstrate how to use AWS Lambda to retrieve data that's in a DynamoDB database. We're doing this project to learn how to set up the data tier of a web app and connect that with a Lambda function too (logic tier).

Tools and concepts

Services we used in this project were Amazon DynamoDB and AWS Lambda. Key concepts we learnt include Lambda functions, adding table items, testing Lambda functions, how to select the right permission policy, writing custom inline policies

Project reflection

This project took us approximately 1 hour and 15 minutes including demo time. The most challenging part was writing our own inline policy (super new experience). It was most rewarding to see the right data pop up when we ran a function test.

We did this project today to learn about the data tier of a three-tier architecture. This project definitely met our goals as we learnt how to connect it to AWS Lambda (logic tier) as well!



Project Setup

To set up our project, we created a database using DynamoDB. DynamoDB is a NoSQL database, so it is very fast at retrieval and flexible for data storage. The partition key is `userId`, which means the key identified for each data is its `userId`.

In our DynamoDB table, we added a piece of data. DynamoDB is schemaless, which means that the data we add can come with new attributes, and every piece of data can have its set of attributes without affecting other data.

Attributes

☐ View DynamoDB JSON

```
1 ▼ {  
2   "userId": "1",  
3   "name": "Test User",  
4   "email": "test@example.com"  
5 }  
6
```

AWS Lambda

AWS Lambda is a service that lets us run code without having to manage servers. We're using Lambda in this project to create a function (i.e. a piece of code) that retrieves data from a DynamoDB table.



AWS Lambda Function

My Lambda function has an execution role, which is the set of permissions that it has to interact with other AWS services. By default, the role grants basic Lambda permissions, which are the permissions to write logs with CloudWatch e.g. error logs.

Our Lambda function will retrieve data from a DynamoDB database table. The first half of our code is all about the data retrieval. The second half of the code is all about sending responses back e.g. success messages, item not found, error messages.

The code uses AWS SDK, which is a set of tools that makes it easier for developers to use AWS in their application code. Our code uses SDK to get access to DynamoDB actions i.e. read/retrieve data items.

```
JS index.mjs ×
JS index.mjs > handler
1 import { DynamoDBClient } from "@aws-sdk/client-dynamodb";
2 import { DynamoDBDocumentClient, GetCommand } from "@aws-sdk/lib-dynamodb";
3
4 const ddbClient = new DynamoDBClient({ region: 'us-west-2' }); // Make sure to replace
5 const ddb = DynamoDBDocumentClient.from(ddbClient);
6
7 async function handler(event) {
8   const userId = String(event.userId); // Make sure to extract userId from the event
9   const params = {
10     TableName: 'UserData',
11     Key: { userId }
12   };
13
14   try {
15     const command = new GetCommand(params);
16     const data = await ddb.send(command); // Log the raw response from DynamoDB
17     console.log("DynamoDB Response:", JSON.stringify(data));
18     const { Item } = data;
19     if (Item) {
20       console.log("User data retrieved:", Item);
21       return Item;
22     }
23   } catch (error) {
24     console.error("Error retrieving user data:", error);
25     return null;
26   }
27 }
```




Function Testing

To test whether our Lambda function works, we wrote a function test in the Test tab in the Lambda console. The test is written in JSON. If the test is successful, we'd see the correct piece of data in JSON when we query for `userId=1`.

The test displayed a 'success' because the function itself ran with no errors i.e. no syntax or dependency issues. But, the function's response was actually an error because our Lambda function is retrieving data from DynamoDB without permissions.

```
Lambda > Functions > RetrieveUserData1

✔ Executing function: succeeded (logs [🔗])
▼ Details

The area below shows the last 4 KB of the execution log.

{
  "name": "AccessDeniedException",
  "$fault": "client",
  "$metadata": {
    "httpStatusCode": 400,
    "requestId": "83QPD2M5I02EVP8SPF49LE42QJVV4KQNS05AEMVJF66Q9ASUAAJG",
    "attempts": 1,
    "totalRetryDelay": 0
  },
  "__type": "com.amazon.coral.service#AccessDeniedException",
  "message": "User: arn:aws:sts::471112976395:assumed-role/RetrieveUserData1-role-vcuqp9ev/RetrieveUserData1 is not authorized to perform: dynamodb:GetItem on resource: arn:aws:dynamodb:us-west-2:471112976395:table/UserData because no identity-based policy allows the dynamodb:GetItem action"
}
```



Function Permissions

To resolve the AccessDenied error, we are providing our Lambda function with the permissions to perform `*GetItem*` on a DynamoDB table. This is because `GetItem` is the action that Lambda is currently blocked from doing.

There were four DynamoDB permission policies we could choose from, but we didn't pick `AWSLambdaDynamoDBExecutionRole` or `AWSLambdaInvocation-DynamoDB` because they relate to DynamoDB streams i.e. live updates of table items that have changed.

We also didn't pick `AmazonDynamoDBFullAccess` because it would give our Lambda function the permission to do too many things with the table. `ReadOnlyAccess` was the right choice because it secures our table from unauthorized access through Lambda.

Other permissions policies (1/1074)

		Filter by Type
dynamodb		All types
☐ Policy name	☐ Type	
☐  AmazonDynamoDBFullAccess	AWS managed	
☒  AmazonDynamoDBReadOnlyAccess	AWS managed	
☐  AWSLambdaDynamoDBExecutionRole	AWS managed	
☐  AWSLambdaInvocation-DynamoDB	AWS managed	



Final Testing and Reflection

To validate our new permission settings, we re-tested our Lambda function. The results were a piece of data and its three attributes (email, name, userId) because that is the exact data that we inputted in our database at the start of this project!

Web apps are a popular use case of using Lambda and DynamoDB. For example, we could use Lambda to retrieve product information, user profiles, content (e.g. blogs or news articles) based on user queries or actions in our web app.

✅ Executing function: succeeded ([logs](#) )

▼ Details

The area below shows the last 4 KB of the execution log.

```
{  
  "email": "test@example.com",  
  "name": "Test User",  
  "userId": "1"  
}
```

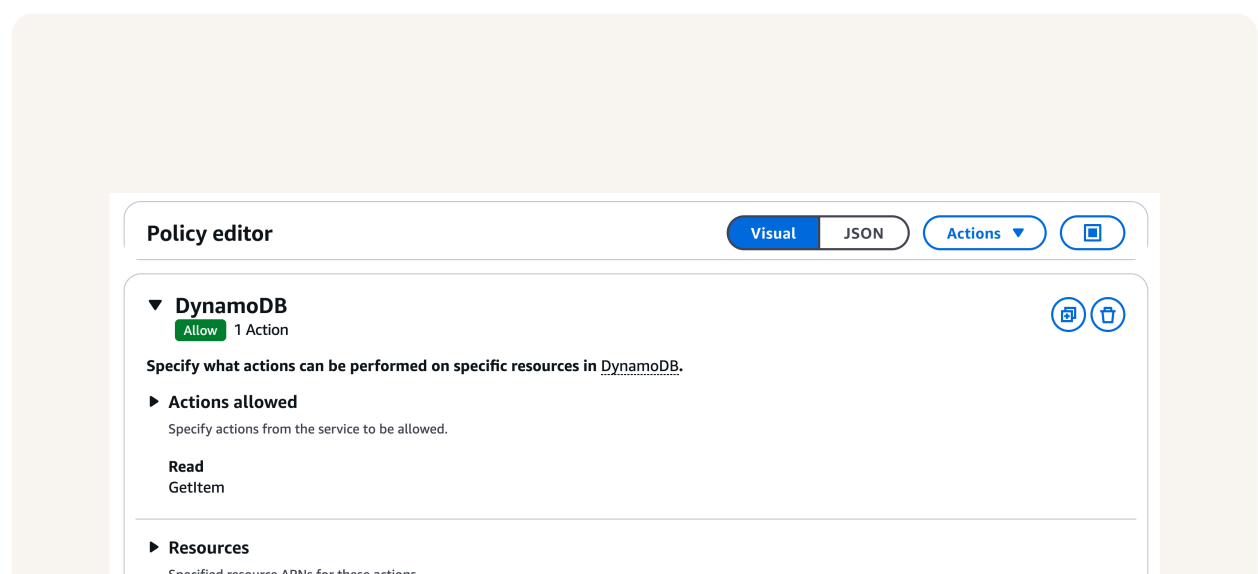


Enhancing Security

For our extension, we challenged ourselves to replace the permission policy we granted our Lambda function previously. Instead of the AWS managed policy, we'll make sure it can only access UserData. This will enhance DynamoDB's security.

To create the permission policy, we used the visual editor because this was a more user friendly and guided option for creating our custom policy. This involved checkboxes and dropdowns that we picked, and the policy was written in the background.

When updating a Lambda function's permission policies, you could risk losing access to resources/actions it needs to work. We validated that our Lambda function still works by re-testing it after updating its permission policies - and yes, still work





NextWork.org

Everyone should be in a job they love.

Check out nextwork.org for
more projects

